

Supplementary Material for “A Mechanically Verified Translation of $\text{MTL}_{0,\infty}$ into Timed Büchi Automata”

Elie Fares, Jean-Paul Bodeveix, Hussain Al-Aqrabi, and Azar Salami



This supplement gives the full reset-policy and residual-obligation proof, the omitted reset-completion arguments, and auxiliary clock arithmetic. The Coq source files used by the development are included in the accompanying `proof/` directory. Cross-references point to the main paper.

1 OMITTED PROOFS FOR THE TBA CONSTRUCTION

1.1 Reset Completion

Proof of Lemma 4.2. The extension ρ' is clock-consistent by construction. Consider a preservation clock x . If the cube taken at position i contains $\text{unch}(x)$, then neither ρ nor ρ' resets x at i ; otherwise ρ' resets it. Hence ρ' resets x wherever ρ does, and the clock recurrence (9) gives, by induction on positions, $\text{val}_{\rho'}(i, x) \leq \text{val}_{\rho}(i, x)$. The literals over x are upper bounds, so remain true. Dually, ρ' resets a restart clock only where the cube contains $\text{rst}(x)$, hence only where ρ resets it. Therefore $\text{val}_{\rho'}(i, x) \geq \text{val}_{\rho}(i, x)$, and the lower-bound literals remain true. Action literals and the Büchi condition are unchanged, so the same run is accepting after completion (Coq lemma `complete_complete`).

1.2 Exactness of Relaxation and Reset Completion

Proof of Proposition 4.3. For soundness, a run of the completed TBA gives a run of the relaxed automaton on the propositional word of its clock-consistent extension. The reset and preservation markers agree with the completed reset set, so the run satisfies every retained literal. By language preservation of relaxation and the backend contract, this word satisfies $\mathcal{T}(f)$. The soundness direction of Theorem 5.1 gives $w, 0 \models_{\text{MTL}} f$.

For completeness, Theorem 5.1 supplies a clock-consistent extension of w satisfying $\mathcal{T}(f)$. The backend contract accepts its propositional word, and relaxation preserves that run. The reset-completion lemma then supplies an extension accepted by the completed TBA. The mechanized theorem is `MTL_to_TBA_correct_with`.

2 DETAILS OF THE CORRECTNESS PROOF

This section gives the parts of the proof of Theorem 5.1 that the article only outlines (Section 5.1); the auxiliary lemmas follow in Section 3.

2.1 Semantic Setting

Let

$$w = (a_0, t_0)(a_1, t_1)(a_2, t_2) \cdots$$

be a timed word, where the timestamps are nonnegative, strictly increasing, and time-divergent. We write

$$\Delta_i = t_{i+1} - t_i > 0, \quad \text{elap}_w(i, j) = t_j - t_i.$$

The MTL satisfaction relation is written $w, i \models_{\text{MTL}} f$.

The translation uses an *extended word* ρ containing the same timed word together with clock values and reset decisions. In the Coq development, clocks are not indexed by syntax-tree paths. The clocks of a formula f are its timed subformulas:

$$\text{Clock}(f) = \{F \mid F \text{ is a primitive timed subformula of } f\},$$

a finite type (`Clock f`), and $\text{clock_of}(f, F)$ is the formula F viewed as an element of $\text{Clock}(f)$. Extended words, LTL formulas over the extended alphabet, the translation, and the automata are all indexed by f , so every clock they mention is, by its type, a timed subformula of f . Thus, a primitive timed formula F denotes one logical clock independently of where it occurs in the syntax tree. If two different paths locate the same syntactic formula, they therefore locate the same clock. For a clock $x \in \text{Clock}(f)$ we write

$$\text{val}_{\rho}(i, x) \in \mathbb{R}_{\geq 0}, \quad \text{reset}_{\rho}(i, x) \in \{\text{true}, \text{false}\}.$$

We write $\text{ew_base}(\rho)$ for the underlying timed word of ρ ; the predicate $\text{same_base}(\rho, w)$ states that $\text{ew_base}(\rho) = w$.

Clock consistency imposes the recurrence

$$\text{val}_{\rho}(i+1, x) = \begin{cases} \Delta_i, & \text{if } \text{reset}_{\rho}(i, x) = \text{true}, \\ \text{val}_{\rho}(i, x) + \Delta_i, & \text{otherwise,} \end{cases} \quad (1)$$

together with nonnegativity of all clock values. Thus, a reset at position i makes the clock measure the time elapsed since t_i . (In the notation of Section 2.2, $\Delta_i = \delta_{i+1}$.)

The LTL satisfaction relation $\rho, i \models_{\text{LTL}} \varphi$ is the standard one, with the atoms interpreted on ρ : an action a holds iff $a_i = a$; $\rho, i \models_{\text{LTL}} (x \sim d)$ iff $\text{val}_{\rho}(i, x) \sim d$; $\rho, i \models_{\text{LTL}} \text{rst}(x)$ iff $\text{reset}_{\rho}(i, x) = \text{true}$; and $\rho, i \models_{\text{LTL}} \text{unch}(x)$ iff $\text{reset}_{\rho}(i, x) = \text{false}$.

Paths are nevertheless retained for structural traversal. For a formula f located at path π , $\mathcal{T}_{\pi}(f)$ denotes its LTL

translation, and $\text{node}(\text{root}, \pi)$ returns the subformula at that path. The auxiliary function $\text{clock_at}(\text{root}, \pi)$ first locates the formula and then applies $\text{clock_of}(\text{root}, \cdot)$. Consequently,

$$\begin{aligned} \text{node}(\text{root}, \pi_1) = F = \text{node}(\text{root}, \pi_2) \\ \implies \text{clock_at}(\text{root}, \pi_1) = \text{clock_at}(\text{root}, \pi_2). \end{aligned}$$

This is the content of the Coq lemma `identical_nodes_share_clock`.

2.2 Derived Ordinary Timed Operators

The source notation contains ordinary and hatted timed operators, but only the hatted ones are primitive in the Coq datatype. The ordinary operators are the definitions of Eqs. (1)–(4), whose agreement with the standard pointwise semantics is Theorem 2.1. The upper-bounded identities separate the case in which the current position already supplies the relevant endpoint. For lower bounds, well-formedness requires $d > 0$, so a qualifying endpoint is necessarily strictly later than the current position; strict upper bounds also require $d > 0$. Since these equations are Coq definitions (`MUle`, `MUlt`, `MUge`, `MUgt`, and their Release counterparts), ordinary timed operators are reduced to Boolean structure plus one hatted timed primitive before the timed proof cases are considered. This is why the structural proof has eight timed branches rather than sixteen.

2.3 Clock Sharing for Identical Subformulas and Repeated Obligations

The Coq development establishes two levels of sharing.

Sharing between different syntax-tree occurrences. As explained in the semantic setting, two occurrences of the same primitive timed subformula share one clock even at different paths; this is built into the semantics of the verified translation, not a post-processing optimization. In addition, the lemma `identical_nodes_share_canonical_trace` shows that the two occurrences have the same canonical value/reset trace.

Sharing between repeated run-time activations. A single timed subformula may also be activated repeatedly under $\square$, U , or R , so several obligations can overlap even though they use one shared clock. The canonical reset policy resolves this competition by keeping an old reference when it dominates newer activations and restarting the clock only in the dual cases.

For upper-bounded hatted Until, suppose an older activation started at i_0 and a newer one at i_1 , with $i_0 \leq i_1 < j$. If j is a valid endpoint for the older obligation, then

$$t_j - t_{i_1} \leq t_j - t_{i_0} \leq d$$

and the prefix required by the newer activation is a suffix of the older prefix. Therefore keeping the oldest still-useful clock reference cannot lose the newer obligation. In the mechanized proof this reasoning is carried by the residual predicate used in `complete_UhatLe`.

For lower-bounded hatted Release, an older unresolved reference has a clock value at least as large as any newer one and therefore reaches the lower threshold no later. The residual predicate `RRes` (Section 2.6) records precisely

the part of the Release obligation that remains after some clock age has already accumulated. Its one-step preservation property is isolated in the Coq lemma `rge_residual_shift` and used by `complete_RhatGe`.

The dual families use the opposite policy. For $\hat{U}_{\geq d}$ and $\hat{R}_{\leq d}$, a newly active obligation may reset the shared clock. The stronger LTL continuation (the continuation Γ of lower-bounded Until, Section 2.4, and the weak-until branch of upper-bounded Release) ensures that older semantic obligations remain represented after the restart. Each of these arguments holds for the strict comparison of the same family, with the strict residual predicates of Section 2.6.

2.4 Translation of the Primitive Timed Operators

Consider a primitive hatted timed formula F at syntax-tree path π , with left and right subformulas p and q . Let

$$x = \text{clock_of}(F), \quad \bar{P} = \mathcal{T}_{\pi_0}(p), \quad \bar{Q} = \mathcal{T}_{\pi_1}(q),$$

and write

$$H = \text{unch}(x), \quad R = \text{rst}(x), \quad \alpha \mathcal{W} \beta \equiv (\alpha \mathcal{U} \beta) \vee G\alpha.$$

Let $\preceq \in \{\leq, <\}$ and $\succeq \in \{\geq, >\}$, with the complementary comparisons $\preceq^c$ and $\succeq^c$ of Section 3.2. For lower-bounded hatted Until define

$$\Theta = (x \succeq d) \wedge (\bar{P} \mathcal{U} \bar{Q}), \quad \Gamma = (\bar{P} \mathcal{U} \Theta) \vee (G\bar{P} \wedge G\bar{F}\bar{Q}),$$

and for the Release cases define

$$\begin{aligned} D &= (x \preceq^c d) \vee (\bar{P} \wedge \bar{Q}), \quad K = (x \succeq^c d) \wedge H, \\ \Lambda &= (\bar{P} \mathcal{R} \bar{Q}) \vee ((x \preceq^c d) \wedge \bar{P}). \end{aligned}$$

The complete primitive timed translation mechanized in Coq consists of eight clauses, of four shapes:

$$\begin{aligned} \mathcal{T}_{\pi}(p \hat{U}_{\preceq d} q) &= X(((x \preceq d) \wedge H \wedge \bar{P}) \\ &\quad \mathcal{U} ((x \preceq d) \wedge \bar{Q})), \end{aligned}$$

$$\mathcal{T}_{\pi}(p \hat{U}_{\succeq d} q) = R \wedge X\Gamma,$$

$$\mathcal{T}_{\pi}(p \hat{R}_{\preceq d} q) = R \wedge X(\bar{Q} \mathcal{W} D),$$

$$\mathcal{T}_{\pi}(p \hat{R}_{\succeq d} q) = X(K \mathcal{W} \Lambda).$$

The ordinary timed operators have no translation clauses of their own. Their Coq definitions unfold into Boolean combinations containing the corresponding hatted operator, after which the homomorphic Boolean translation and the clauses above apply automatically.

All primitive formulas start logical evaluation after the current position, which accounts for the X in every clause. The current timestamp remains the clock reference, as required by the hatted semantics.

2.5 Auxiliary Lemmas

The proof uses elementary facts about LTL (the semantics of GF , Eq. (4), and the unfolding of Until and weak until) and clock arithmetic: the value of a preserved clock accumulates the elapsed time (Lemma 3.1), a reset clock measures the time elapsed since its last reset, and the clock tests therefore bound the elapsed time from above and from below (Eqs. (7) and (8)), with the strict variants of Section 3.3. These lemmas and their proofs are given in Section 3.

2.6 Residual Obligations and the Canonical Extension

Soundness can reason about an arbitrary clock-consistent extension. Completeness, however, must construct a reset trace that realizes the translation. Following the canonical extended-word construction of CASAAL [1], the proof does this using a deterministic canonical policy and residual predicates, adapted here to formula-keyed shared clocks.

For upper-bounded Until, define

$$\begin{aligned} \text{URes}(i, v, d, p, q) &\iff \\ \exists j \geq i. v + \text{elap}_w(i, j) &\leq d \wedge w, j \models_{\text{MTL}} q \\ \wedge \forall k \in [i, j). w, k &\models_{\text{MTL}} p. \end{aligned}$$

The value v represents time accumulated before the current position. Thus URes means that an older upper-bounded Until obligation is still able to reach a valid q endpoint.

For lower-bounded Release, define

$$\begin{aligned} \text{RRes}(i, v, d, p, q) &\iff \\ \forall j \geq i. d \leq v + \text{elap}_w(i, j) &\implies \\ (w, j \models_{\text{MTL}} q \vee \exists k \in [i, j). w, k &\models_{\text{MTL}} p). \end{aligned}$$

Here the residual represents a Release obligation that started before i and has already accumulated clock age v .

For the strict primitives, the proof uses the strict counterparts

$$\begin{aligned} \text{URes}^<(i, v, d, p, q) &\iff \\ \exists j \geq i. v + \text{elap}_w(i, j) &< d \wedge w, j \models_{\text{MTL}} q \\ \wedge \forall k \in [i, j). w, k &\models_{\text{MTL}} p, \\ \text{RRes}^>(i, v, d, p, q) &\iff \\ \forall j \geq i. d < v + \text{elap}_w(i, j) &\implies \\ (w, j \models_{\text{MTL}} q \vee \exists k \in [i, j). w, k &\models_{\text{MTL}} p). \end{aligned}$$

They are the Coq predicates `ult_residual` and `rgt_residual`; the one-step update of the latter is `rgt_residual_shift`.

The reset decision is selected by matching on the *formula-keyed clock* itself, so the canonical policy has one case per hatted primitive; the strict cases use the strict residual predicates $\text{URes}^<$ and $\text{RRes}^>$ defined above.

Definition 2.1 (Canonical reset policy and canonical extension). For a clock $x = F$, a position i , and a current clock value v , the reset decision $\text{reset}^*(F, i, v)$ is defined as follows:

- $F = p \hat{\text{U}}_{\leq d} q$: the clock is preserved (not reset) iff $v \leq d$, $\text{URes}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$;
- $F = p \hat{\text{U}}_{< d} q$: the clock is preserved iff $v < d$, $\text{URes}^<(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} q$;
- $F = p \hat{\text{U}}_{\geq d} q$ or $F = p \hat{\text{R}}_{\geq d} q$: the clock is reset iff $w, i \models_{\text{MTL}} F$;
- $F = p \hat{\text{R}}_{> d} q$: the clock is preserved iff $v < d$, $\text{RRes}(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$;
- $F = p \hat{\text{R}}_{\geq d} q$: the clock is preserved iff $v \leq d$, $\text{RRes}^>(i, v, d, p, q)$, and $w, i \not\models_{\text{MTL}} p$.

The canonical extension $\rho^* = \text{canonical_ext}(\text{root}, w)$ has base word w and, writing $u_i = \text{val}_{\rho^*}(i, x)$, clock values

$$u_0 = 0, \quad u_{i+1} = \begin{cases} \Delta_i, & \text{if } \text{reset}^*(x, i, u_i), \\ u_i + \Delta_i, & \text{otherwise,} \end{cases}$$

and reset decisions $\text{reset}_{\rho^*}(i, x) = \text{reset}^*(x, i, u_i)$.

This is the Coq definition `reset_policy` with `canonical_val`. The decisions depend on the future of w (through the residual predicates and the satisfaction of F), so ρ^* is a non-constructive witness defined with classical logic; it is used only to prove completeness.

The side conditions $w, i \not\models_{\text{MTL}} q$ and $w, i \not\models_{\text{MTL}} p$ are needed for the canonical policy. If the older $\hat{\text{U}}_{\leq d}$ obligation is discharged by q at the current position i , nothing is required of the clock after i , and the clock is reset for the newer activations; preserving it there could make a newer obligation miss its bound. Symmetrically, a p at position i discharges an older $\hat{\text{R}}_{\geq d}$ obligation. For $\hat{\text{U}}_{\geq d}$ and $\hat{\text{R}}_{\leq d}$, restarting at every position where F holds keeps the newest reference, and the Γ and weak-until continuations of the translation keep older obligations represented.

The policy depends only on the clock $x = F$ and on w , not on the syntax-tree path; this is why one decision simultaneously governs all syntax-tree occurrences of the same timed formula. Ordinary timed operators require no additional policy cases because they unfold to these hatted primitives. By construction,

$$\text{same_base}(\rho^*, w) \quad \text{and} \quad \text{clock_consistent}(\rho^*). \quad (2)$$

The residual predicates above are the mechanized form of the clock-reuse principle stated informally in Section 2.3. They allow the canonical policy to carry an older reference forward without allocating a fresh clock for each activation.

2.7 Soundness of the Primitive Timed Translations

For each primitive hatted timed constructor F , the local soundness lemma has the form

$$\rho, i \models_{\text{LTL}} \mathcal{T}_\pi(F) \implies \text{ew_base}(\rho), i \models_{\text{MTL}} F, \quad (3)$$

assuming clock consistency and the induction hypotheses for the immediate subformulas. No canonical reset-policy assumption appears in (3).

Upper-bounded hatted Until. The translated LTL Until starts at $i + 1$ and provides an endpoint $j > i$. At position i itself the translation does not constrain the clock, but the recurrence (1) together with nonnegativity gives $\text{val}_\rho(i + 1, x) \geq \Delta_i$ whether or not x is reset at i . Along the intermediate positions (i, j) the shared clock is unchanged, so Lemma 3.1 applied on $[i + 1, j)$ yields $\text{val}_\rho(j, x) \geq \Delta_i + \text{elap}_w(i + 1, j) = \text{elap}_w(i, j)$, and the endpoint guard $x \leq d$ gives

$$\text{elap}_w(i, j) \leq d.$$

The induction hypotheses turn the translated prefix $\bar{P}$ into p and the endpoint $\bar{Q}$ into q , matching the semantics of $p \hat{\text{U}}_{\leq d} q$. For $\hat{\text{U}}_{< d}$, the strict guard $x < d$ gives $\text{elap}_w(i, j) < d$ in the same way (`sound_UhatLt`).

Lower-bounded hatted Until. The translation resets x at i and follows Γ . In the finite branch, a future state satisfies $x \geq d$ together with $\bar{P} \text{ U } \bar{Q}$. By (8), the endpoint is at least d time units after i , and the nested Until supplies the required p -prefix and q -endpoint. In the second branch, $G\bar{P} \wedge GF\bar{Q}$ holds. Then p holds forever and q recurs infinitely often; time divergence guarantees a q occurrence at physical distance at least d . For $\hat{\text{U}}_{> d}$, the strict threshold $x > d$ gives a strict lower bound in the finite branch, and the fairness

branch uses the strict argument of Section 3.3 (`sound_UhatGt`).

Upper-bounded hatted Release. The translation resets the shared clock and requires

$$\overline{Q} \, W \, ((x > d) \vee (\overline{P} \wedge \overline{Q})).$$

Before the weak-until endpoint, q holds at every strictly future position. If the endpoint is $\overline{P} \wedge \overline{Q}$, p discharges the Release obligation; if it is $x > d$, the upper-bounded window has ended. For $\hat{R}_{<d}$, the window ends at $x \geq d$ (`sound_RhatLt`).

Lower-bounded hatted Release. The translation requires

$$((x < d) \wedge \text{unch}(x)) \, W \, ((\overline{P} \, R \, \overline{Q}) \vee ((x < d) \wedge \overline{P}))$$

from $i + 1$. While the left side holds, the clock is preserved and remains below d , so those positions are still before the lower threshold. At the endpoint either an ordinary untimed Release takes over or p discharges the obligation before the threshold is reached. For $\hat{R}_{>d}$, the pre-threshold test is $x \leq d$ (`sound_RhatGt`).

There are no separate ordinary timed soundness lemmas. After unfolding Eqs. (1)–(4), their correctness follows from the Boolean cases of the structural induction and the eight hatted lemmas above.

2.8 Completeness under the Canonical Reset Policy

The converse local result is proved for the canonical extension:

$$w, i \models_{\text{MTL}} F \implies \rho^*, i \models_{\text{LTL}} \mathcal{T}_\pi(F).$$

The difficult point is that all occurrences of the same primitive timed formula share one clock and each occurrence may itself be activated repeatedly. Thus several semantic obligations can compete for the same clock trace. The canonical reset policy and the residual predicates are designed to resolve this interaction.

Upper-bounded hatted Until. Choose the first q -position among the MTL witnesses. The proof maintains, for every position n with $i < n \leq j$ up to that endpoint j , the invariant

$$\text{val}_{\rho^*}(n, x) + \text{elap}_w(n, j) \leq d.$$

It need not hold at the activation position i itself (the policy may reset there because the old value is useless); it is established at $i + 1$ in both the reset and the preservation case. If the policy preserves the clock, Definition 2.1 guarantees an older reference whose residual URes is still satisfiable strictly after the current position; if it resets, the new value is consistent with the current activation. Hence every intermediate position satisfies the clock guard required by the translated Until and the first q -position supplies its endpoint. This is the content of the mechanized lemma `complete_UhatLe`. The strict case maintains the strict invariant $\text{val}_{\rho^*}(n, x) + \text{elap}_w(n, j) < d$ with the residual $\text{URes}^<(\text{complete_UhatLt})$.

Lower-bounded hatted Until. At every position where F holds, the canonical policy resets the shared clock (Definition 2.1). The proof propagates an active obligation until a position satisfies

$$x \geq d \quad \text{and} \quad \overline{P} \, U \, \overline{Q},$$

which yields the finite branch of Γ . If no such position is reached, the active-obligation invariant implies that p remains continuously true, and since every later position again carries a pending obligation that requires a later q , the proposition q recurs infinitely often. This gives

$$\text{G}\overline{P} \wedge \text{GF}\overline{Q},$$

the second branch of Γ . This is formalized by `complete_UhatGe`, and by `complete_UhatGt` for the strict threshold $x > d$.

Upper-bounded hatted Release. The canonical policy resets the clock at every position where F holds. The proof follows the currently active nontrivial Release obligation. Until it is discharged, the clock measures elapsed time from the relevant start and q continues to hold. The weak until terminates either because $p \wedge q$ discharges the Release or because $x > d$, meaning that the upper-bounded window has ended. This is the role of `complete_RhatLe`, and of `complete_RhatLt` for the strict window end $x \geq d$.

Lower-bounded hatted Release. Here the invariant is

$$\text{RRes}(n, \text{val}_{\rho^*}(n, x), d, p, q).$$

By Definition 2.1, the shared clock is preserved exactly while $x < d$, the residual holds, and p does not occur; in that case the residual shifts from (i, v) to

$$(i + 1, v + \Delta_i).$$

The Coq lemma `rge_residual_shift` establishes this one-step update. A converse lemma on the preservation policy recovers the residual information needed at the next position, so the left side of the weak until remains valid until either an ordinary untimed Release is established or p occurs while the clock is still below d . This yields `complete_RhatGe`. The strict case uses $\text{RRes}^>$, the pre-threshold test $x \leq d$, and the shift lemma `rgt_residual_shift` (`complete_RhatGt`).

As with soundness, ordinary timed operators need no additional completeness lemmas: unfolding their definitions reduces them to Boolean structure and one of the eight hatted primitive cases above.

2.9 Structural Equivalence at Every Syntactic Occurrence

The local timed lemmas are assembled with the homomorphic Boolean and untimed temporal cases into the following stronger induction theorem.

Theorem 2.2 (Canonical correctness at every occurrence). *Let root be well formed and suppose*

$$\text{node}(\text{root}, \pi) = f.$$

Then for every position i ,

$$w, i \models_{\text{MTL}} f \iff \rho^*, i \models_{\text{LTL}} \mathcal{T}_\pi(f),$$

where $\rho^ = \text{canonical_ext}(\text{root}, w)$.*

Proof idea. The proof is by structural induction on the formula f . Atomic propositions, Boolean connectives, Next, untimed Until, and untimed Release follow directly from their semantics and the induction hypotheses. Since the

ordinary timed operators are Coq definitions, they have already unfolded into these structural cases. The only primitive timed branches are therefore the eight hatted constructors. In each such branch, the forward direction invokes the corresponding completeness lemma from Section 2.8, while the reverse direction invokes the corresponding soundness lemma from Section 2.7. The node lemmas locate the two children, and well-formedness of the located formula provides the bound and recursive hypotheses. $\square$

This is stronger than a root-only statement and is important in the mechanized proof. The induction retains the path in order to locate the current subformula and its children, but the path is not the clock key. If two paths locate the same primitive timed formula, both branches refer to the same formula-keyed clock.

2.10 Proof of the Correctness Theorem

We now prove Theorem 5.1.

Proof. We prove the two implications separately.

($\Rightarrow$) *MTL to clocked LTL.* Assume $w, 0 \models_{\text{MTL}} f$. Choose the canonical extension

$$\rho = \text{canonical_ext}(f, w).$$

By (2), ρ has the same base timed word as w and is clock-consistent. Applying Theorem 2.2 at the root path $[]$ and position 0 gives

$$\rho, 0 \models_{\text{LTL}} \mathcal{T}(f).$$

Therefore the required extension exists.

($\Leftarrow$) *Clocked LTL to MTL.* Assume there exists an arbitrary ρ such that

$$\begin{aligned} \text{same_base}(\rho, w), \quad \text{clock_consistent}(\rho), \\ \rho, 0 \models_{\text{LTL}} \mathcal{T}(f). \end{aligned}$$

This extension need not be canonical, so completeness cannot be used. Instead, a structural induction on f uses the policy-independent soundness lemmas of Section 2.7 and obtains

$$\text{ew_base}(\rho), 0 \models_{\text{MTL}} f.$$

Finally, $\text{same_base}(\rho, w)$ rewrites the underlying timed word $\text{ew_base}(\rho)$ to w , yielding $w, 0 \models_{\text{MTL}} f$. $\square$

Scope of the mechanized result. Theorem 5.1 covers the MTL formula, its clocked-LTL encoding, and clock-consistent extensions. Corollary 5.3 extends the result, through relaxation and reset completion, to the timed automaton built from any backend automaton satisfying the stated propositional-language contract. Initial clock values remain existential in this theorem. Agreement with conventional zero initialization, the parser and external backend interface, and output printing are outside the result proved here.

3 AUXILIARY LEMMAS OF THE CORRECTNESS PROOF

This section records the semantic and arithmetic facts used in the proof of Theorem 5.1.

3.1 Basic LTL Semantic Lemmas

For every LTL formula A and position i ,

$$\begin{aligned} \rho, i \models_{\text{LTL}} GA &\iff \forall j \geq i, \rho, j \models_{\text{LTL}} A, \\ \rho, i \models_{\text{LTL}} FA &\iff \exists j \geq i, \rho, j \models_{\text{LTL}} A, \\ \rho, i \models_{\text{LTL}} GFA &\iff \forall n \geq i \exists j \geq n, \rho, j \models_{\text{LTL}} A, \end{aligned} \quad (4)$$

$$\begin{aligned} \rho, i \models_{\text{LTL}} A WB &\iff \left(\exists j \geq i. \right. \\ &\quad \rho, j \models_{\text{LTL}} B \\ &\quad \wedge \forall k \in [i, j). \\ &\quad \left. \rho, k \models_{\text{LTL}} A \right) \\ &\quad \wedge \forall j \geq i. \rho, j \models_{\text{LTL}} A. \end{aligned} \quad (5)$$

These are Coq lemmas `LG_semantics`, `LF_semantics`, `LGF_semantics`, and `LW_semantics`. Time divergence adds the following useful consequence: if $d \geq 0$ and GFB holds from i , some future B occurs at physical distance at least d from i (Coq: `GF_far`).

3.2 Clock Arithmetic

Clock consistency turns symbolic guards into elapsed-time inequalities.

Lemma 3.1 (Accumulation under preservation). *If x is not reset at any position in $[i, j)$, then*

$$\text{val}_\rho(j, x) = \text{val}_\rho(i, x) + \text{elap}_w(i, j).$$

Since clock values are nonnegative, $\text{elap}_w(i, j) \leq \text{val}_\rho(j, x)$. Thus an upper guard $x \leq d$ at j entails

$$\text{elap}_w(i, j) \leq d. \quad (7)$$

An upper guard $x < d$ gives the strict version.

Lemma 3.2 (Clock value after a reset). *If x is reset at i and $j \geq i + 1$, then $\text{val}_\rho(j, x) \leq \text{elap}_w(i, j)$. If it is not reset again before j , equality holds.*

A later reset only makes the clock younger. Hence a lower guard gives

$$x \geq d \implies d \leq \text{val}_\rho(j, x) \leq \text{elap}_w(i, j), \quad (8)$$

and $x > d$ entails a strict lower bound. The Coq development proves these facts as `preserve_accumulate`, `after_reset_value_le_elapsed`, and `reset_preserve_value_eq_elapsed`.

3.3 Strict Bounds

The eight hatted constructors are the primitive proof families. Ordinary timed operators have already been reduced by Eqs. (1)–(4). Strict comparisons remain strict through each translation and proof case. In particular, preservation gives $\text{elap}_w(i, j) \leq \text{val}_\rho(j, x)$, so $x < d$ implies $\text{elap}_w(i, j) < d$. After a reset, $\text{val}_\rho(j, x) \leq \text{elap}_w(i, j)$, so $x > d$ implies $\text{elap}_w(i, j) > d$.

The only strict case requiring more than direct guard arithmetic is the fairness branch of $\hat{U}_{>d}$. Time divergence gives a future q at distance at least d from position $i + 1$; strict timestamp increase gives $t_{i+1} > t_i$, hence the selected q is at distance strictly greater than d from i . This is the argument formalized in Coq lemma `sound_UhatGt`.

4 COQ RESULT MAP

The paper’s principal statements correspond to these definitions and theorems in the included Coq development.

Proof source. The Coq definitions of `EncodingCorrect_proved` and `MTL_to_TBA_correct_with` are in `EncodingCorrect_Shared_Clock_Derived_Strict_Direct_Proof.v`.

Core source. The definitions of `T_uses_every_clock` and `complete_complete` are in `MTL_to_TBA_Shared_Clock_Derived_Strict_Direct_Core.v`.

The theorem with suffix `_with` assumes the backend correctness contract as an argument. The development includes no project-specific assumption in that theorem beyond this explicit contract; the standard library assumptions are recorded by Coq’s `Print Assumptions` output in the proof source.

REFERENCES

- [1] G. Li, P. G. Jensen, K. G. Larsen, A. Legay, and D. B. Poulsen, “Practical controller synthesis for $MTL_{0,\infty}$,” in *Proceedings of the 24th ACM SIGSOFT International SPIN Symposium on Model Checking of Software*, ser. SPIN 2017. New York, NY, USA: Association for Computing Machinery, 2017, pp. 102–111. [Online]. Available: <https://doi.org/10.1145/3092282.3092303>